AXIOM: A Shared Graph-Attention Controller for Scalable Multi-Agent 5G Slice Admission

Manoj Prasad Kunasegran, Wai Leong Pang*, Swee King Phang

School of Engineering, Faculty of Innovation and Technology, Taylor's University, Subang Jaya, 47500, Selangor, Malaysia.

*Corresponding author(s). E-mail(s): waileong.pang@taylors.edu.my;

Abstract

Fifth generation (5G) network slicing must admit heterogeneous requests across cooperating base stations under strict per-slice guarantees. Multi-agent learning can adapt these decisions to changing user demand and load. However, state-of-the-art research rarely evaluates learned multi-agent controllers against trivial policies or on independent seeds. We present AXIOM, an Adaptive Cross-cell Integrity-aware Open Radio Access Network Multi-agent slice-admission controller. Its graph-attention encoder, shared by every gNodeB (gNB), feeds one decision head per slice, keeping its size fixed as clusters grow. A pre-registered protocol with both checks compares it with single-agent and independent deep Q-networks (DQN), multi-agent proximal policy optimisation (MAPPO), value decomposition, and tuned rules, under static and coupled time-varying load. At 7, 19, and 37 gNBs, AXIOM keeps Ultra-Reliable Low-Latency Communication (URLLC) and enhanced Mobile Broadband (eMBB) service-level compliance above 93%, while every learned baseline stays below 84%. At 37 gNBs, AXIOM leads the best learned baseline by 35.0 to 36.5 points. There it serves 99.8% of URLLC demand with a backlog delay 7 to 44 times shorter than the learned baselines, using only 7,782 parameters against 64,770 and 682,058 for the deep Q-networks. Under coupling, AXIOM earns 27% to 36% more reward than MAPPO at 3 and 19 gNBs, and 39% more than a static shedding rule at 3 gNBs. When load rises 10% to 20% past a tuned rule's conditions, AXIOM overtakes that rule by up to 8.7 reward per step. AXIOM is therefore a viable alternative for dynamic, scalable, self-optimising slice admission in next-generation 5G networks.

Keywords: Network Slicing, Slice Admission Control, Multi-Agent Reinforcement Learning, Graph Neural Networks, Federated Learning, Open RAN

M. P. Kunasegran, orcid.org/0009-0005-3169-1873. W. L. Pang, orcid.org/0000-0001-8407-5648. S. K. Phang, orcid.org/0000-0002-7877-8766.

1 Introduction

Network slicing virtualises the radio access network (RAN) to serve Ultra-Reliable Low-Latency Communication (URLLC), enhanced Mobile Broadband (eMBB), and massive Machine-Type Communication (mMTC), each under its own Service Level Agreement (SLA). Our earlier simulation studies showed that deep reinforcement learning, including a deep Q-network (DQN), can protect URLLC SLAs at a single congested gNodeB (gNB) [16] and can balance load across a gNB cluster [17]. However, those controllers used a centralised approach, whereas a real deployment spans many gNBs that share spectrum planning and a cluster-wide load-balance objective. Consequently, admission control across such a cluster becomes a multi-agent problem. In this research, we ask whether per-gNB agents that share a view of the whole cluster can decide better than a single centralised agent or independent per-gNB agents. Fig. 1 places our research question within the broader system context.

This paper introduces AXIOM, an acronym for Adaptive Cross-cell Integrity-aware Open Radio Access Network Multi-agent slice-admission controller. Here the Open Radio Access Network (O-RAN) denotes a softwarised and open-source RAN architecture, and integrity-aware means that every reported result passes checks of evaluation integrity. One encoder, shared by every gNB, feeds a separate Q-head for each slice, under centralised training and decentralised execution. The encoder is a Graph Attention Network (GAT) [38], and the per-slice heads, each a small multi-layer perceptron (MLP), carry AXIOM's advantage as the cluster grows. Three gaps found in our prior systematic review of 67 core studies motivate pairing AXIOM with a stricter evaluation, as Section 2 details. No reviewed study checks whether its outcome metric can score a degenerate policy, such as one that accepts every request, as well as a correct

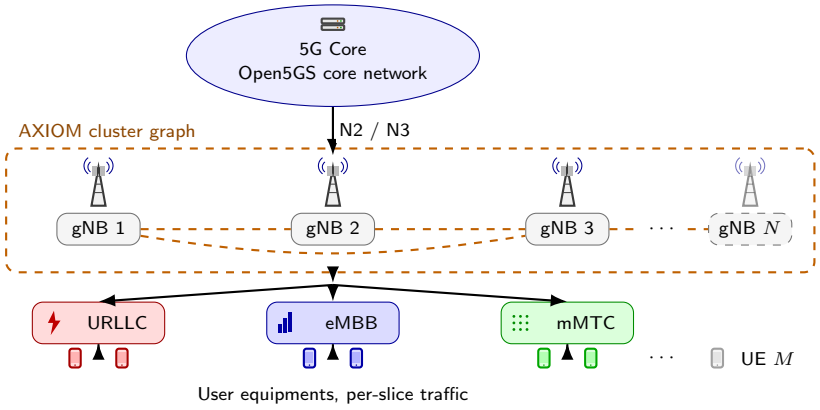


Fig. 1 Conceptual system view. A cluster of N gNBs, coordinated by the shared AXIOM encoder and backhauled to a 5G core, serves M user equipments (UEs) across three slices. AXIOM is evaluated offline, and the live rig runs one gNB

one, which blocks just enough low-priority requests to protect the priority slices. In addition, none replicates a headline finding on an independent seed sample, and none tests whether a live testbed reaches the contention that its offline claims assume. A seed fixes the random draws of one run, such as its starting weights and the traffic it trains and is tested on, so a finding that also holds on seeds never used before is unlikely to be an artefact of one sample.

We evaluate AXIOM in two offline simulators of a gNB cluster. Firstly, we consider a simple environment (Env A) with a static load, where shedding every mMTC request is close to optimal. Secondly, we consider Env B, which adds time-varying load and inter-cell coupling, so shedding one fixed slice is no longer optimal. We then compare AXIOM with a single-agent deep Q-network (DQN-SA), independent per-gNB deep Q-networks (DQN-I), multi-agent proximal policy optimisation (MAPPO), and tuned threshold rules. We also grow the cluster to 7, 19, and 37 gNBs under three layouts. Further campaigns test federated training, differential privacy, and network disruption. The contributions of this paper are as follows.

1. AXIOM, the Adaptive Cross-cell Integrity-aware O-RAN Multi-agent slice-admission controller, with a shared cluster encoder and decentralised per-slice Q-heads, whose parameter count does not grow with the cluster and whose per-slice heads, rather than attention, carry its advantage at scale.
2. Env A, a shared-budget environment with static load, built after a trivial-policy check showed that accepting every request beat every learned policy in our original simulator. In Env A, AXIOM keeps priority-slice compliance above 93% at 7, 19, and 37 gNBs, while both DQN baselines and MAPPO stay below 84%.
3. Env B, a more demanding environment with time-varying load and inter-cell coupling. There AXIOM leads DQN-SA, DQN-I, and MAPPO at three and nineteen gNBs and beats a static shedding rule at three, while a tuned threshold rule sets a bar that no learned policy reaches under its tuning conditions.
4. An evaluation protocol for learned slice admission control that combines trivial-policy checks, per-slice correctness metrics, independent-seed replication, tuned rule baselines, and a pre-registered analysis.

The remainder of this paper is organised as follows. Section 2 reviews related work, Section 3 formulates the admission problem, and Section 4 presents the AXIOM architecture. Section 5 describes the experimental setup and evaluation safeguards, and Section 6 reports the results. Finally, Section 7 discusses the limitations, and Section 8 concludes the paper.

2 Related Work

Our prior systematic review [18] screened 6,286 records down to 67 core studies from 2024 to 2026 on machine-learning-driven, user-centric distributed network slicing. The review organised them around three enabling pillars, namely topology-aware graph representation, distributed multi-agent control, and privacy-preserving federation. Each pillar is well developed on its own, but they are rarely combined under realistic constraints, and never jointly validated in a live deployment.

Studies from the review's core sample show the state of each pillar. Learned admission control uses sequential twin actor-critics [24] and digital-twin-accelerated deep reinforcement learning [34]. Multi-agent control spans sample-efficient multi-agent DQNs for inter-slice orchestration [8], scalable frameworks for dynamic RAN slicing [20], and cooperative agents for energy-efficient slicing [25]. A graph neural network with meta-learning makes bandwidth allocation scalable and transferable [11], and Ahmadi et al. [2] study topology generalisation through attention. Federated deep reinforcement learning has been applied to splitting virtual network functions [4] and to slice mobility [22]. Two core studies combine two of the three pillars, graph attention with multi-agent learning for task offloading [23] and a graph neural network with multi-agent learning for resource orchestration [6]. Closest to our evaluation concern, Tsampazi et al. [37] compare many deep reinforcement learning agents for O-RAN in an automated pipeline, a comparison that our trivial-policy and independent-seed checks extend. None of these core studies checks its metric against a trivial policy or replicates a finding on independent seeds.

Table 1 Closest prior work, as described in each paper's title and abstract. A dagger marks a core study of our review [18]

Work	Problem	Multi-agent	Graph encoder	Federated
Shao et al. [30]	Slicing resource management in dense cells	Yes	Attention	Not described
Sulaiman et al. [32]	Joint slicing and slice admission in cloud RAN	Yes	Not described	Not described
Ahmadi et al. [2]†	RAN and edge-computing slicing with admission control	Yes	Attention	Not described
Zangooei et al. [41]	SLA-constrained RAN slicing in Open RAN	Yes	Not described	Not described
Mohajer et al. [23]†	SLA-aware task offloading and service orchestration	Yes	Attention	Not described
Bouroudi et al. [6]†	Energy-efficient network-function placement	Yes	Graph neural network	Not described
AXIOM	Per-slice admission across a gNB cluster	Yes	Graph attention	Federated averaging, optional differential privacy

Apart from the core studies that Table 1 marks, the works cited in the rest of this section lie outside the review's 67 core studies. They add earlier foundations, studies published in this journal, and further recent work. AXIOM also sits inside the broader O-RAN effort to run learned control loops over standard interfaces, which Polese et al. [27] survey and which ColO-RAN [26] and Tsampazi et al. [36] exercise on large experimental platforms. Learned slice admission predates O-RAN, from market-driven admission [5] to reinforcement learning for RAN slicing [29], and Khani et al. [13] survey it for cloud RAN. OACR^2 [19] uses request forecasting to maximise the infrastructure provider's long-term revenue. AXIOM instead protects priority-slice SLAs, the objective of an operator bound by strict per-slice guarantees. Cooperative agents usually train by value factorisation such as QMIX [28] or with a centralised critic as in MAPPO [40], and constrained multi-agent learning has been applied to Open

RAN slicing at scale [41]. Recent work in this journal applies actor-critic learning [15] and double deep Q-networks [12] to related problems. Other studies in it address delay-aware allocation [3], federated learning [14], and multi-agent relay selection [31]. None of these studies checks its metric against a trivial policy or replicates on independent seeds. Unlike these studies, AXIOM shares one encoder across agents and gives every slice its own decision head, which Section 6.3 shows matters as the cluster grows.

Two further works come close on a single axis of this problem. Shao et al. [30] show that graph attention strengthens inter-agent cooperation. Sulaiman et al. [32] are the closest prior art to joint slicing and admission control, yet neither study checks for degenerate policies or replicates on independent seeds. Table 1 sets AXIOM beside the six closest works from both groups, and none of them combines admission control, a graph encoder, and federated training as AXIOM does.

Robustness studies [9, 35] test disruptive conditions, as our disruption campaign does, yet neither checks whether its robustness metric is trustworthy or replicates a finding. Zhang et al. [42] and Yasin et al. [39] are the closest precedents for periodic synchronisation and for federated learning with differential privacy, but both secure the controller rather than characterise a control policy's own privacy-utility trade-off. Gong et al. [10] combine federated training with multi-agent control, two of the three pillars. Like every work above, each of these evaluates its own outcome metric once, on one seed or run.

3 System Model and Problem Formulation

This section formulates slice admission across a gNB cluster as a decentralised decision problem and describes the two simulators that evaluate it. We consider a cluster of N gNBs, from three in the main campaigns to 37 in the scaling study. Each gNB serves URLLC, eMBB, and mMTC under a shared per-gNB Physical Resource Block (PRB) budget B. At each control step every gNB holds zero or more pending requests, each tagged with a slice, and its agent accepts or rejects each one, conditioned on the cluster state. Each decision moves the slice's PRB-ratio ceiling, the control that the xApp interface of OpenAirInterface (OAI) exposes. The binary action matches our single-gNB work, so any coordination benefit comes from the architecture. At three gNBs the cluster is treated as fully connected, a modelling choice, and the scaling study adds ring and hex-grid layouts. The model works at the PRB level and abstracts the radio channel. Channel quality changes how many PRBs a slice needs for the same traffic, and the model folds this into the per-step demand noise. A channel-aware extension that maps the signal-to-interference-plus-noise ratio to PRB demand is left to future work.

The step reward follows our earlier formulations [16, 17]. It combines a per-slice service term net of SLA-violation penalties with a congestion charge on admitted volume and a load-imbalance penalty. For slice k, ω_k is the priority weight and R_k the per-accept reward. Its admitted count is n_k and its violation penalty λ_k, and $\mathbf{1}[v_k]$ flags an SLA violation in that step. With these terms, the reward of step t is then

$$r_t = \sum_k \big(\omega_k R_k\, n_k - \lambda_k\, \mathbf{1}[v_k]\big) \;-\; \kappa\, C_t \|\mathbf{a}_t\|_1 \;-\; \beta\,(1 - F_t), \tag{1}$$

where $\|\mathbf{a}_t\|_1$ is the total admitted this step and C_t the cluster-mean congestion level. The congestion coefficient is κ, and $F_t \in [0,1]$ is the min-over-max PRB-utilisation fairness ratio with load-balance weight β. The violation flag v_k follows from a continuous SLA margin m_k. The correctness-aware metrics of Section 5.1 also build on this margin. We take it as the worse of the queue and loss channels,

$$m_k = \min\Big(1 - \tilde{q}_k,\ 1 - \tfrac{\ell_k}{\ell_k^{\max}}\Big), \qquad v_k \Leftrightarrow m_k \le 0, \tag{2}$$

with $\tilde{q}_k$ the slice backlog relative to its budget, and ℓ_k and $\ell_k^{\max}$ its loss proxy and loss budget. The weights are set so that, under congestion, the intended policy is differentiated shedding, which keeps accepting URLLC and eMBB and sheds mMTC.

Admission is therefore a decentralised partially observable Markov decision process. At step t the state s_t comprises every gNB's queues, offered demand, and slice ceilings. Agent i observes its own feature row $\mathbf{x}_i$, which the shared encoder combines with the other rows of $\mathbf{X}_t$. For each pending request it chooses $a \in \{\text{accept}, \text{reject}\}$, the shared-budget dynamics move the cluster to s_{t+1}, and every agent receives the common reward r_t of Eq. (1). The agents jointly seek a policy π that maximises the discounted return over an episode of T=60 steps,

$$\max_{\pi}\ \mathbb{E}_{\pi}\Big[\textstyle\sum_{t=0}^{T-1} \gamma^t r_t\Big], \qquad \gamma = 0.95. \tag{3}$$

Each gNB has a budget of B=100 PRBs in both simulators. The nominal shares are 30, 60, and 10 PRBs for URLLC, eMBB, and mMTC. Each slice's mean offered demand is 105% of its share, scaled by a fixed per-gNB multiplier m_i between 0.9 and 1.0. This puts a gNB's mean demand at about 95% to 105% of its budget, before per-step noise and backlog. When admitted demand exceeds B, the slices share it in proportion to demand, so every admitted mMTC request takes resources from URLLC and eMBB. Unserved demand carries forward as backlog within an episode. Env B keeps all of this and adds two mechanisms. Each gNB's mean demand follows $0.95\,(1 + 0.3\sin(2\pi(s+\psi_i)/60))$ times its nominal shares, again scaled by m_i, with s the step and ψ_i a phase redrawn per gNB and episode. As a result, demand swings between about 60% and 124% of the budget. A gNB's usable budget also shrinks with its neighbours' previous load, $B_i(t) = B\,(1 - 0.15\,\bar{u}_{\mathcal{N}(i)}(t-1))$, a first-order model of inter-cell interference over the same adjacency graph the encoder reads. In Env B shedding every mMTC request leaves URLLC compliance at 62.5% at three gNBs, whereas a rule that also sheds eMBB when the URLLC queue builds keeps it at 98.9%. Per-slice compliance counts a slice as compliant only when every gNB meets it, which saturates in Env B at nineteen gNBs, so there we also report per-gNB compliance. Fig. 2 traces one control step of both simulators, and Table 2 lists the remaining simulation and training parameters.

Table 2 Simulation and training parameters not stated in the text, shared by both environments. Slice values are listed as URLLC, eMBB, mMTC

Parameter	Value	Parameter	Value
Floor and cap, PRB ratio	5 to 33, 10 to 66, 5 to 11	Priority weight ω_k	12.0, 3.5, 0.3
Violation penalty λ_k	10.0, 5.0, 2.5	Per-accept reward R_k	1.0 for every slice
Latency budget	15, 45, 65 ms	Loss budget	0.8%, 2.0%, 3.5%
Congestion κ, balance β	1.5, 1.0	Episode	60 steps of 5 s, queues emptied at start
Requests per step	N, uniform slice and gNB, at most $4N$ pending	Demand noise, gate step	$0.005B$ per step, 5 PRB-ratio points per decision
Value-based learners	Adam, learning rate 10^{-3}, ϵ from 1.0 to 0.05 at $\times 0.985$ per episode, replay 10,000, minibatch 16, warm-up 32, target sync every 10 episodes	AXIOM network	Two-layer, four-head GAT, hidden and output 16, raw-feature skip, per-slice Q-heads of hidden 32, Huber loss, gradient clip 1.0
DQN baselines	Two hidden layers of 128, squared-error loss	MAPPO	Learning rate 10^{-3}, 10 epochs, minibatch 64, clip 0.2, entropy 0.01, advantage $\lambda = 0.95$

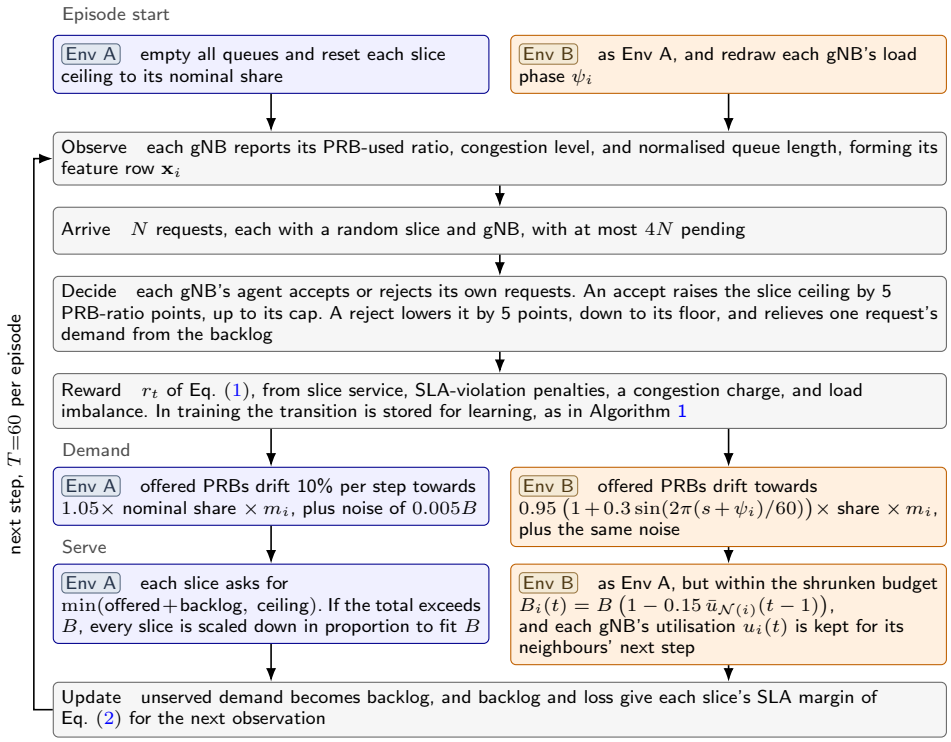


Fig. 2 One control step of both simulators, repeated for T=60 steps per episode. Shared steps span the full width, while the split steps show where Env A and Env B differ

4 The AXIOM Architecture

This section presents the AXIOM architecture, its training rule, and its federated variant. AXIOM differs from its DQN baselines in what each policy sees and shares. AXIOM trains one encoder over the whole cluster from every agent's experience and reads out one Q-head per slice. DQN-SA is one controller over a flattened joint state, and DQN-I gives each gNB its own learner over local features only. The two baselines are therefore ablations that remove the shared encoder.

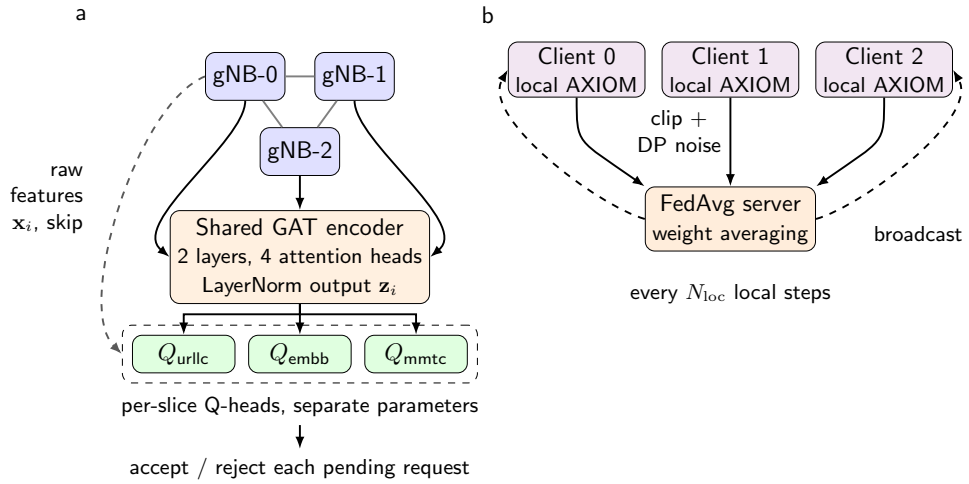


Fig. 3 AXIOM architecture. a, A shared graph-attention encoder produces one embedding per gNB, and per-slice Q-heads read that gNB's embedding and raw features. b, The federated variant, each gNB a client synchronised by periodic federated averaging with optional privacy noise

4.1 Shared Encoder, Decentralised Q-Heads

Fig. 3a shows the AXIOM architecture and its data flow. At every step each gNB forms a feature row $\mathbf{x}_i$ of its PRB-used ratio, congestion level, and normalised queue length, and the rows stack into $\mathbf{X} \in \mathbb{R}^{N \times d}$. A two-layer, four-head GAT over the adjacency graph maps $\mathbf{X}$ to one normalised embedding $\mathbf{z}_i$ per gNB, and Section S4 of Online Resource 1 (OR1) gives its attention equations. Every agent's loss updates this encoder, which is centralised training, while each agent decides its own pending requests from its own row $\mathbf{z}_i$, which is decentralised execution.

Two design choices in AXIOM matter most at scale. One small MLP per slice gives each slice its own parameters, so URLLC's large priority weight cannot swamp mMTC's smaller learning signal in a shared head, which Section 6.3 confirms at nineteen gNBs. Each node's raw features are also concatenated onto its normalised embedding before the heads, $Q_{\phi_k}([\mathbf{z}_i, \mathbf{x}_i])$, so a head sees absolute queue and congestion levels. From here on, $\mathbf{z}_i$ denotes that concatenated input to the heads. This skip

connection and a learning rate of 10^{-3}, both chosen on Env B pilot seeds before any reported run, are the only changes from the original AXIOM design.

In training, every pending request at gNB i, slice k is its own temporal-difference sample, and Algorithm 1 gives the full training loop. For a request with immediate reward r and next-state embedding $\mathbf{z}'_i$, the target y at discount γ and the minibatch loss are

$$y = r + \gamma\,(1-\eta)\max_{a'} Q_{\phi_k^-}(\mathbf{z}'_i)[a'], \qquad \mathcal{L} = \frac{1}{|\mathcal{M}|}\sum_{(i,k,a)\in\mathcal{M}} \ell_\delta\Big(Q_{\phi_k}(\mathbf{z}_i)[a] - y\Big), \tag{4}$$

where $Q_{\phi_k^-}$ is slice k's target Q-head, $\eta \in \{0,1\}$ the flag that marks an episode's last step, $\mathcal{M}$ the minibatch of requests, and ℓ_δ the Huber loss. Section 6.3 shows that a summed per-step target works as well, so the per-request scheme is a simplification rather than a source of AXIOM's advantage.

Algorithm 1 Centralised training and decentralised execution of AXIOM

Require: encoder f_θ, per-slice Q-heads $\{Q_{\phi_k}\}$, targets $f_{\theta^-}, \{Q_{\phi_k^-}\}$, buffer $\mathcal{D}$, adjacency $\mathbf{A}$, discount γ, sync period T_s

for each step t, observing joint node features $\mathbf{X}_t$ **do**
  $\mathbf{E}_t \leftarrow f_\theta(\mathbf{X}_t, \mathbf{A})$ {centralised, one row per gNB}
  for all pending requests at gNB i, slice k **do**
    $a \leftarrow \arg\max_a Q_{\phi_k}([\mathbf{E}_t[i], \mathbf{X}_t[i]])[a]$, ϵ-greedy {decentralised, own row only}
  end for
  step env, store $(\mathbf{X}_t, \mathbf{X}_{t+1}, \{(i,k,a)\}, r_t, \eta_t)$ in $\mathcal{D}$
  if $|\mathcal{D}| \geq$ warmup **then**
    sample minibatch $\mathcal{M}$, then $\mathbf{E}_\mathcal{M} \leftarrow f_\theta(\mathbf{X}_\mathcal{M}, \mathbf{A})$, $\mathbf{E}'_\mathcal{M} \leftarrow f_{\theta^-}(\mathbf{X}'_\mathcal{M}, \mathbf{A})$
    per request, $y \leftarrow r + \gamma(1-\eta)\max_{a'} Q_{\phi_k^-}([\mathbf{E}'_\mathcal{M}[i], \mathbf{X}'_\mathcal{M}[i]])[a']$
    $\mathcal{L} \leftarrow$ mean per-request Huber loss, never summed per step, then update $\theta, \{\phi_k\}$ with gradient norm clipped at 1.0
  end if
  every T_s episodes, $\theta^- \leftarrow \theta$, $\phi_k^- \leftarrow \phi_k$
end for

4.2 Federated Training with Differential Privacy

Fig. 3b shows the federated variant, in which each gNB holds a private copy of the network, trains only on its own requests, and every N_loc local steps sends its weights for federated averaging (FedAvg) [21]. Each client can add differential privacy through a differentially private stochastic gradient descent (DP-SGD) step [1], which clips its minibatch gradient and adds Gaussian noise of multiplier σ. The privacy loss is accounted for with zero-concentrated differential privacy [7]. Section S4 of OR1 gives the clipping, the privacy accounting, and its assumptions.

5 Experimental Setup

This section describes the campaigns, the baselines, and the statistical tests, followed by the safeguards that protect every reported result. Every campaign uses 300 training and 50 evaluation episodes per seed, where a seed fixes one network initialisation and one environment stream, and each seed is evaluated on a held-out stream. The three-gNB campaigns use 30 seeds, 900 to 929, with a disjoint replication sample, 1000 to 1009. The scaling campaigns use 15 seeds per layout at seven gNBs, 30 at nineteen, and at thirty-seven 30 on the hex layout and 15 on each other layout. MAPPO [40] is a published multi-agent baseline with a shared actor and a centralised critic, trained on-policy for the same 300 episodes on the same reward. Its learning rate, epochs, and minibatch size were tuned on pilot seeds, because untuned it collapsed in four of five.

Rules need no learning, and accept-all simply admits every request. The oracle rule of Env A sheds mMTC whenever its gNB's congestion level reaches a threshold τ, chosen on training streams only. The selected value is τ=0, so the rule sheds every mMTC request. We call it the oracle rule because it is the best rule of its family on training streams. It is a strong reference rather than a proven optimum, since across our Env A samples learned policies exceeded it in 28 of 450 seed-level evaluations, by at most 0.40 reward per step. In Env B we add two tuned rule families, each chosen on training streams over a grid of thresholds and evaluated on the held-out streams. The local family sheds mMTC on the gNB's own congestion or priority-slice queue and sheds eMBB on its URLLC queue, while the neighbour-aware family also reads the mean congestion of the gNB's neighbours. Two further runs repeat the DQN baselines at a learning rate of 10^{-4}.

All learned policies share $\gamma = 0.95$, the training budget, and the exploration schedule of Table 2, and Section S4 of OR1 gives the design of the sensitivity studies.

All confidence intervals (CIs) are 95% bootstrap percentile intervals over 10,000 resamples. Paired comparisons use the Wilcoxon signed-rank test, and each family of comparisons against AXIOM is Holm-corrected for multiple testing. The test drops exactly tied pairs, so where two policies make identical decisions on many seeds, as AXIOM and the oracle rule do in Env A, a comparison rests on the remaining seeds.

The settings, seeds, and four hypotheses of the final campaign were fixed in a pre-registration committed to the project's version-control history before any of its runs, rather than lodged with an external registry. Every later campaign was pre-registered the same way. The Env A baselines that this campaign reuses come from an earlier rerun whose plan was committed one minute after that rerun was launched and before any of its results were read. Section S4 of OR1 lists the reused results. H1 is that AXIOM beats both trivial rules in Env B, and H2 compares it with every learned baseline and tuned rule in Env B. H3 asks whether graph attention beats a per-node MLP encoder in Env B at seven gNBs, and H4 compares AXIOM with every baseline at nineteen gNBs in Env A.

A pre-registered validation screen checks every trained checkpoint on its own training stream before evaluation. Every seed is reported whether or not it passes, and Section S4 of OR1 gives the screen's criteria.

5.1 Evaluation Safeguards

Three safeguards protect the results, and Table 3 lists the failure each one caught. A trivial-policy check evaluates accept-all and one-slice shedding before any learned policy is trusted, and it exposed an earlier simulator in which accepting every request was optimal.

Correctness-aware metrics form the second safeguard of the evaluation. Block precision $\pi = b_{\mathrm{mMTC}}/b$ is the share of a policy's b blocked requests that target mMTC, and a seed that never blocks counts as a collapse rather than a success. Per-slice SLA compliance then shows whether shedding protects URLLC and eMBB, whereas all-slice compliance scores a correct policy and a failed one alike, because correct shedding gives up mMTC's SLA by design.

Finally, every headline comparison is recomputed on an independent, disjoint seed sample. This caught queue backlog carried over between episodes, and every result here comes from the corrected simulator, which empties all queues at the start of each episode.

Table 3 Evaluation failures this paper's safeguards caught during the work

Failure	Missed by	Caught by	Outcome
Inert simulator (§3)	Learned-policy metrics	Accept-all baseline	Shared-budget rerun
Never-blocking policy scored as fine	All-slice compliance	Precision, per-slice SLA	Collapse made visible
Reproduction gap (OR1)	Single-sample claim	Reproduction against replication	Churn claim retracted
Backlog carry-over (§5.1)	Per-seed means	Replication, step traces	Episode reset, rerun
Static AXIOM in Env B (§4.1)	Env A results	Env B pilot seeds	Faster learning rate, skip connection
Screen miscalibrated (§5)	Env A screen design	Pilot screen check	Pre-registered amendment
Scheduler floor (§6.7)	Cross-layer floor	Direct instrumentation	Over 100,000 readings clear

6 Results

We summarise per-slice compliance as priority-slice compliance, the mean of URLLC and eMBB compliance, and call a trained policy collapsed when it accepts every request. Table 4 sets out how the four learned policies stand. Throughout, a reward margin is a difference in mean reward per step, a compliance margin is in percentage points, and a load level is a percentage of the nominal offered demand.

6.1 Static Load, Three gNBs

In the thirty-seed campaign of Table 5, AXIOM never collapses and comes closer to the oracle rule's reward than any other learner. AXIOM earns 1.110 more reward per step than DQN-I, with 95% CI [0.60, 1.65]. It earns 2.685 more than MAPPO, with 95% CI [2.01, 3.35], and both margins hold at Holm-adjusted $p \leq 0.0002$. Its margin

Table 4 How the four learned policies compare. Env A has static load, Env B time-varying load with inter-cell coupling. Priority SLA is the mean of URLLC and eMBB compliance

Property	AXIOM	DQN-SA	DQN-I	MAPPO
Design	Shared encoder, per-slice heads	One flattened controller	One learner per gNB	Shared actor, central critic
Parameters at execution	7,782 at any size	21,250 to 64,770	55,302 to 682,058	5,122
Env A reward, 3 gNBs	12.70, 98.9% of oracle	12.39	11.59	10.01
Env A priority SLA, 7, 19, 37 gNBs	97.1, 97.3, 93.7%	76.2, 65.3, 58.7%	74.6, 56.4, 32.1%	67.7, 60.6, 56.7%
Collapse, Env A, 19 and 37 gNBs	1/90 and 6/60	84/90 and 26/30	3/90 and 3/30	0/90 and 0/30
Env B reward, 3 and 19 gNBs	8.83 and 80.28	7.49 and 74.74	7.79 and 71.20	6.97 and 59.07
Env B priority SLA, 7 gNBs	35.1%	44.3%	39.9%	35.9%

over DQN-SA, 0.311 with 95% CI [−0.01, 0.64], does not survive correction on thirty seeds, at $p = 0.066$. A disjoint replication on seeds 1000 to 1009 and the pooled 40 seeds favour AXIOM more strongly, as Section S5 of OR1 reports, but fresh seeds in Section 6.6 do not reproduce the lead over DQN-SA.

Table 5 Env A, three gNBs, 30 seeds per policy, 95% bootstrap CIs. W/T/L counts the seeds where AXIOM earns more, the same, or less. Precision averages the seeds that block at all. SLA columns give per-slice compliance

Policy	Reward/step	Of oracle reward, %	AXIOM W/T/L	Precision	Collapsed	URLLC	eMBB	mMTC
Oracle rule	12.835 [12.77, 12.91]	100.0	2/22/6	1.000	0/30	1.000	1.000	0.366
AXIOM	12.698 [12.49, 12.86]	98.9	–	0.992	0/30	0.998	0.979	0.389
DQN-SA	12.387 [12.11, 12.63]	96.5	15/8/7	0.990	0/30	0.994	0.941	0.436
DQN-I	11.587 [11.11, 12.02]	90.3	27/0/3	0.962	0/30	0.993	0.865	0.402
MAPPO	10.013 [9.30, 10.73]	78.0	29/1/0	0.707	2/30	0.978	0.576	0.664

6.2 Cluster-Size Scaling

We grow the Env A cluster from three gNBs to seven and nineteen under fully-connected, ring, and hex-grid layouts. A pre-registered extension then adds a thirty-seven-gNB cluster on the same layouts. We test the per-layout gaps on reward and on priority-slice compliance, Holm-correcting the 18 comparisons at each size. Fig. 4 traces the result across sizes, and Section S6 of OR1 gives the per-slice detail.

At seven gNBs, under every layout, AXIOM keeps 17.9 to 29.9 percentage points more priority-slice compliance than each baseline and earns 3.40 to 7.07 more reward per step. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.011$. At nineteen gNBs the gaps widen, and they answer hypothesis H4. AXIOM keeps a pooled priority-slice compliance of 97.3% with 95% CI [95.2%, 99.0%]. Under every layout

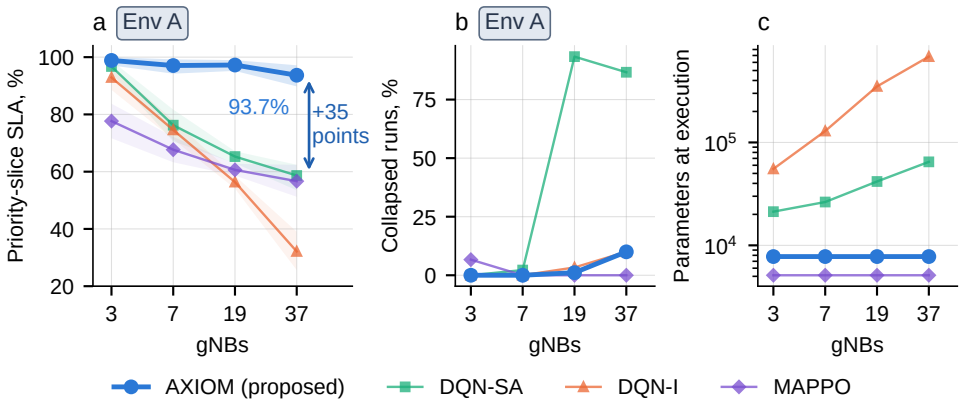


Fig. 4 Env A as the cluster grows, all layouts pooled, with the layout-independent baselines run once per seed at 37 gNBs. a, Priority-slice compliance, mean and 95% CI. b, Share of trained policies that collapse into accept-all. c, Parameters used at execution

AXIOM keeps 31.0 to 42.1 percentage points more compliance than every baseline and earns 10.7 to 18.6 more reward per step, winning 28 to 30 of 30 seeds. All 18 comparisons hold after Holm correction, at adjusted $p \leq 0.0001$.

At thirty-seven gNBs the lead grows further, and AXIOM keeps a pooled priority-slice compliance of 93.7% with 95% CI [89.8%, 97.2%]. Under every layout AXIOM keeps 32.0 to 67.8 percentage points more compliance than every baseline and earns

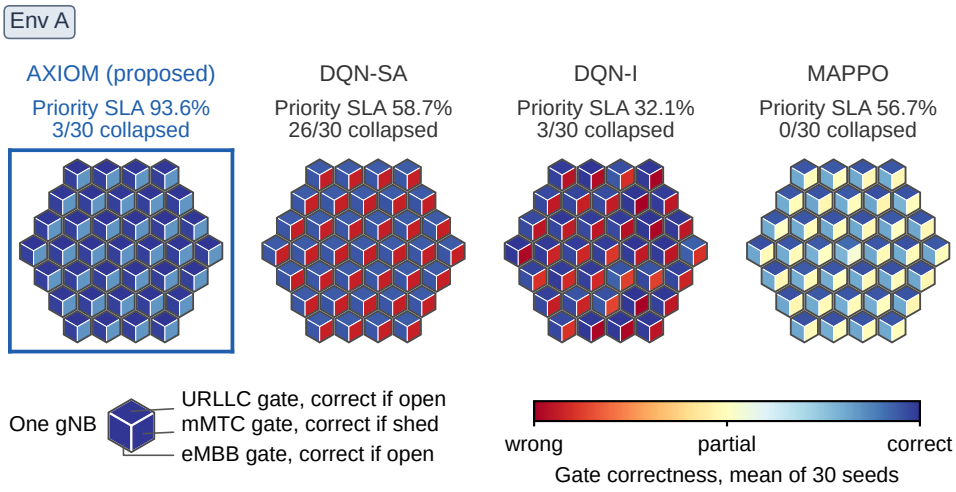


Fig. 5 Decision map of the 37-gNB hex cluster in Env A over 30 seeds. Each hexagon is a gNB at its true position, split into its URLLC, eMBB, and mMTC admission gates and coloured by how correct each gate is

18.3 to 29.5 more reward per step. Here all 18 comparisons hold after Holm correction within their pre-registered families, at adjusted $p < 0.01$. AXIOM collapses more often than at nineteen gNBs, as Table 4 shows, but even its collapsed runs keep 59% to 65%, the level of the best baselines. A pre-registered replication on 20 fresh seeds confirms the result. There AXIOM keeps 95.1% priority-slice compliance against 34.1% to 58.5% for the baselines and leads each on compliance and on reward at Holm-adjusted $p < 0.001$. It collapses in 1 of the 20 seeds, against 18 for DQN-SA. On these clusters AXIOM serves 99.8% of the offered URLLC demand and 99.0% of the eMBB demand, against 94.2% to 97.7% and 85.7% to 97.5% for the learned baselines. Its URLLC delay, the mean backlog over the mean served PRBs in 5 s control periods, is 0.08 with a 95th percentile of 0.42, whereas the baselines reach 0.58 to 3.61 with 95th percentiles of 5.2 to 16.9. Section S8 of OR1 gives throughput and delay in every setting.

Accepting every request is the natural floor at scale. It keeps 70.9%, 67.1%, and 61.9% priority-slice compliance at seven, nineteen, and thirty-seven gNBs. Every baseline falls below it on average at the two larger sizes. AXIOM stays 26 to 32 points above the floor. Its advantage at scale therefore lies in reliably learning the correct shedding policy, the one the static rule encodes, rather than in coordination beyond that rule. The static rule that sheds every mMTC request keeps 99.96% to 99.99% at every size. Where that rule is known, AXIOM comes within 3 points of it up to nineteen gNBs and within 6.3 at thirty-seven, while from seven gNBs up every other learner falls at least 24 points short. Tuning the DQN baselines on pilot seeds does not close the gap, and AXIOM beats each tuned baseline on compliance and on reward at Holm-adjusted $p < 0.001$, by 29.9 to 48.1 points. The published value-decomposition network (VDN) [33] ties AXIOM at three gNBs but trails it by 13.6 points at nineteen, as Section S6 of OR1 reports.

Fig. 5 shows where the gap comes from, gNB by gNB, in the thirty-seven-gNB hex cluster. A gNB's admission gate for a slice is the PRB-ratio ceiling that its accept and reject decisions move between the slice's floor and cap, averaged over every evaluation step of all 30 seeds. AXIOM keeps the URLLC and eMBB gates open at every gNB, at 1.00 and 0.98, and closes the mMTC gate to 0.16 even with its three collapsed seeds included. No baseline closes the mMTC gate below 0.51.

6.3 Ablations and Sensitivity

Four checks test where AXIOM's behaviour comes from, as Section S5 of OR1 details. First, the per-slice Q-heads matter as the cluster grows. A variant with one shared head of matched size ties AXIOM at three gNBs. At nineteen gNBs on the ring layout, however, it collapses in 9 of 15 seeds and keeps only 65.9% priority-slice compliance against AXIOM's 96.8% on the same seeds, losing every seed at Holm-adjusted $p < 0.001$. Second, graph attention adds nothing measurable to AXIOM. A per-node MLP encoder ties AXIOM in Env A. In Env B at seven gNBs it keeps 1.12 and 0.95 more reward per step on the ring and hex layouts and wins 10 of 15 seeds in each. These leads in the pre-registered test H3 miss significance after Holm correction, at $p = 0.061$ and $p = 0.095$. Third, a variant with one joint summed target in place of per-request samples is never significantly different. Fourth, the learning rate matters, and AXIOM

stays within 0.13 of the oracle rule's reward under changed load and slice weights. A 24-configuration MAPPO grid and a seeded DQN-SA still lose to AXIOM in every setting tested.

6.4 Time-Varying Load with Inter-Cell Coupling

Table 6 and Fig. 6 report Env B at three gNBs, where shedding one fixed slice no longer suffices. AXIOM earns 2.495 more reward per step than shedding every mMTC request and 4.476 more than accept-all. Both gaps hold at Holm-adjusted $p < 0.0001$, as hypothesis H1 predicted. It also leads every learned policy, by 1.038 reward per step over DQN-I, 1.337 over DQN-SA, and 1.863 over MAPPO. It also keeps more priority-slice compliance than each, at Holm-adjusted $p < 0.0001$. A tuned rule that also sheds eMBB whenever the URLLC queue passes 0.2 beats every learned policy, earning 0.420 more than AXIOM at $p < 0.0001$.

Table 6 Env B, three gNBs, 30 seeds per policy, 95% bootstrap CIs. Rules above the line, learned policies below it. W/T/L counts the seeds where AXIOM does better, the same, or worse

			AXIOM W/T/L				
Policy	Reward/step	Of tuned rule, %	Reward	Priority SLA	URLLC	eMBB	mMTC
Accept all	4.356 [4.13, 4.60]	47.1	30/0/0	30/0/0	0.446	0.265	0.955
Shed all mMTC	6.337 [6.00, 6.67]	68.5	30/0/0	30/0/0	0.625	0.367	0.371
Tuned local rule	9.252 [9.12, 9.40]	100.0	8/0/22	6/0/24	0.989	0.367	0.390
Tuned neighbour-aware rule	9.041 [8.93, 9.15]	97.7	13/0/17	23/0/7	0.988	0.283	0.901
AXIOM	8.832 [8.64, 9.03]	95.5	–	–	0.958	0.344	0.406
DQN-SA	7.495 [7.20, 7.77]	81.0	29/0/1	26/0/4	0.866	0.284	0.412
DQN-I	7.794 [7.44, 8.13]	84.2	29/0/1	27/0/3	0.910	0.270	0.441
MAPPO	6.968 [6.47, 7.45]	75.3	29/0/1	24/0/6	0.797	0.289	0.579

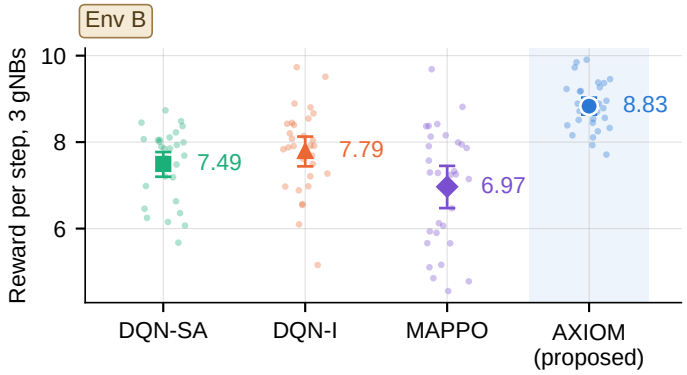


Fig. 6 Env B reward per step of the learned policies at three gNBs, 30 seeds, with per-seed points behind the mean and 95% CI

At seven gNBs the tuned rules beat AXIOM in all 15 seeds under every layout. On the ring layout AXIOM keeps less compliance than DQN-SA, at Holm-adjusted $p = 0.011$. This gap mainly reflects the 300-episode budget, within which 28 of AXIOM's 45 seeds settle on the static rule's policy and never shed eMBB, which none of its 30 seeds does at three gNBs. With 900 episodes for every learner on the ring layout, AXIOM keeps 55.7% priority-slice compliance against 40.5% to 42.9% for the baselines, at Holm-adjusted $p \leq 0.0012$, though still below the tuned rule's 60.5%. After 900 episodes AXIOM rejects eMBB more often as the URLLC queue grows, much as the tuned rule does. At nineteen gNBs the longer budget does not help, and seven of eight seeds keep the static policy after 900 episodes, as Section S7 of OR1 reports.

At nineteen gNBs, where cluster-level compliance saturates, AXIOM earns 5.54 to 21.22 more reward per step than every learned baseline and keeps 5.5 to 10.9 percentage points more per-gNB priority-slice compliance, at Holm-adjusted $p \leq 0.013$. The static rule still earns 1.31 more per step, and the tuned local rule carried over from seven gNBs earns 3.81 more and wins every seed. Fig. 7 maps this per-gNB compliance for the four learned policies. Each hexagon is one gNB, coloured from red at 50% to blue at 85% by the mean share of evaluation steps on which it meets the URLLC and eMBB SLAs. The policies only admit or reject requests where they arrive, so the colours show how well each gNB copes with its own offered load. Every policy keeps the least at the centre gNB, whose load the simulator fixes at the nominal level while it draws every other gNB's from 90% to 100% of it. The six inner gNBs have as many neighbours but trail the edge by only 0.5 to 1.8 points. AXIOM keeps more than every learned baseline at each of the nineteen gNBs, by at least 3.8 points, and by at least 4.3 points on the 15 fresh seeds. The carried-over rule leads AXIOM by 13.0 points at the centre and 6.9 points elsewhere, and by 13.2 and 7.0 points on the fresh seeds.

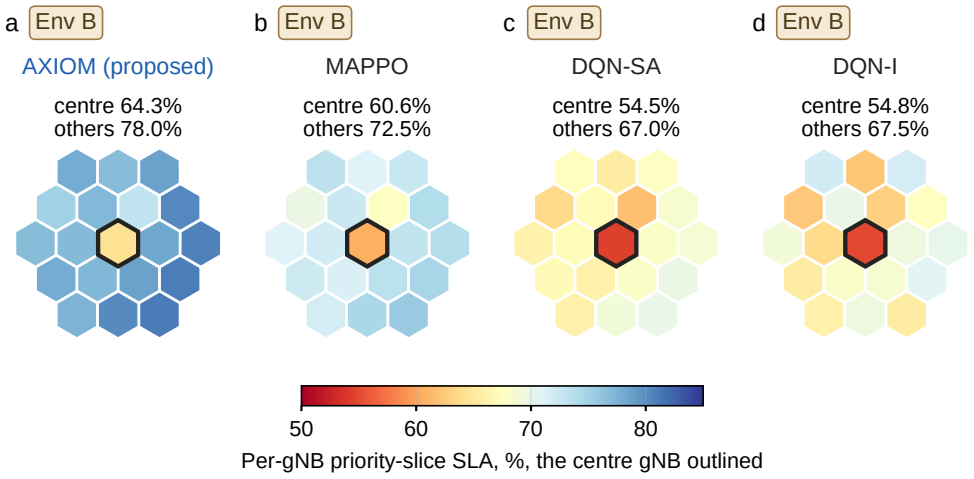


Fig. 7 Per-gNB priority-slice compliance in Env B at nineteen gNBs on the hex layout, mean of 15 seeds. a, AXIOM. b, MAPPO. c, DQN-SA. d, DQN-I. The centre gNB is outlined

The rule's lead holds only under its tuning conditions. When the load rises to 110% to 120% without re-tuning or retraining, AXIOM wins 21 to 23 of 25 seeds at three gNBs, at Holm-adjusted $p \leq 0.0002$. At nineteen gNBs it earns 6.23, 7.55, and 8.67 more reward per step than that rule, winning 14 of 15 seeds at each level at Holm-adjusted $p < 0.001$. A re-tuned rule ties AXIOM up to 115% load and beats it at 120%. Section S7 of OR1 adds uneven per-gNB load and a per-gNB SLA penalty, under which AXIOM still beats every learned baseline but not the rules.

In Fig. 8a, AXIOM stays between 4.2 and 4.7 reward per gNB per step and above 93% priority-slice compliance, while every baseline loses compliance as the cluster grows. In Env B, panel c, every policy loses compliance as the cluster grows. AXIOM earns the most reward per gNB and keeps the most compliance at three and nineteen gNBs, and it earns about as much per gNB as DQN-SA at seven, where DQN-SA keeps more compliance. That gap reflects the training budget, as shown above. Panels b and d place six measures per environment on one scale. In Env A AXIOM leads on five of the six, while MAPPO leads on collapse resistance because it never collapses at thirty-seven gNBs. In Env B, AXIOM leads on four, and DQN-SA leads on both measures at seven gNBs. Panels e and f show how the policies cope when the load moves. In Env A at nineteen gNBs AXIOM stays within 5.3 points of the static rule as offered load rises from 105% to 120% of the nominal shares, and every baseline stays below AXIOM at each load. In Env B AXIOM trails the tuned rule at the rule's own load, by 0.14 to 0.55 reward per gNB, but leads it by 0.10 to 0.46 once the load rises 5% to 20%, with neither retrained nor re-tuned. At nineteen gNBs that rule is the one carried over from seven gNBs.

6.5 Model Complexity

Table 7 sets out the cost of each policy. AXIOM's shared encoder keeps its parameter count fixed as the cluster grows, while the DQN baselines grow with it. Its greedy decisions on one CPU thread stay well inside the 5 s control period of the live testbed. Against MAPPO, the cheapest learner, the extra cost buys the most where the environment is dynamic. In Env B AXIOM earns 27% more reward per step than MAPPO at three gNBs and 36% more at nineteen gNBs. A 24-configuration tuning grid for MAPPO did not close the gap in any setting where it was tested.

Table 7 Model complexity at 3, 7, 19, and 37 gNBs. Parameters used at execution, greedy inference time per control step on one CPU thread, and median training time per seed at 3, 19, and 37 gNBs, on the ring layout for AXIOM

Policy	Parameters	ms per step	Training s
AXIOM	7,782 at every size	1.04 / 1.98 / 4.79 / 9.89	624 / 892 / 1182
DQN-SA	21,250 / 26,370 / 41,730 / 64,770	0.61 / 1.42 / 3.95 / 8.02	85 / 202 / 358
DQN-I	55,302 / 129,038 / 350,246 / 682,058	0.59 / 1.38 / 3.73 / 8.08	181 / 1054 / 2088
MAPPO	5,122 at every size	0.19 / 0.20 / 0.22 / 0.27	73 / 256 / 495

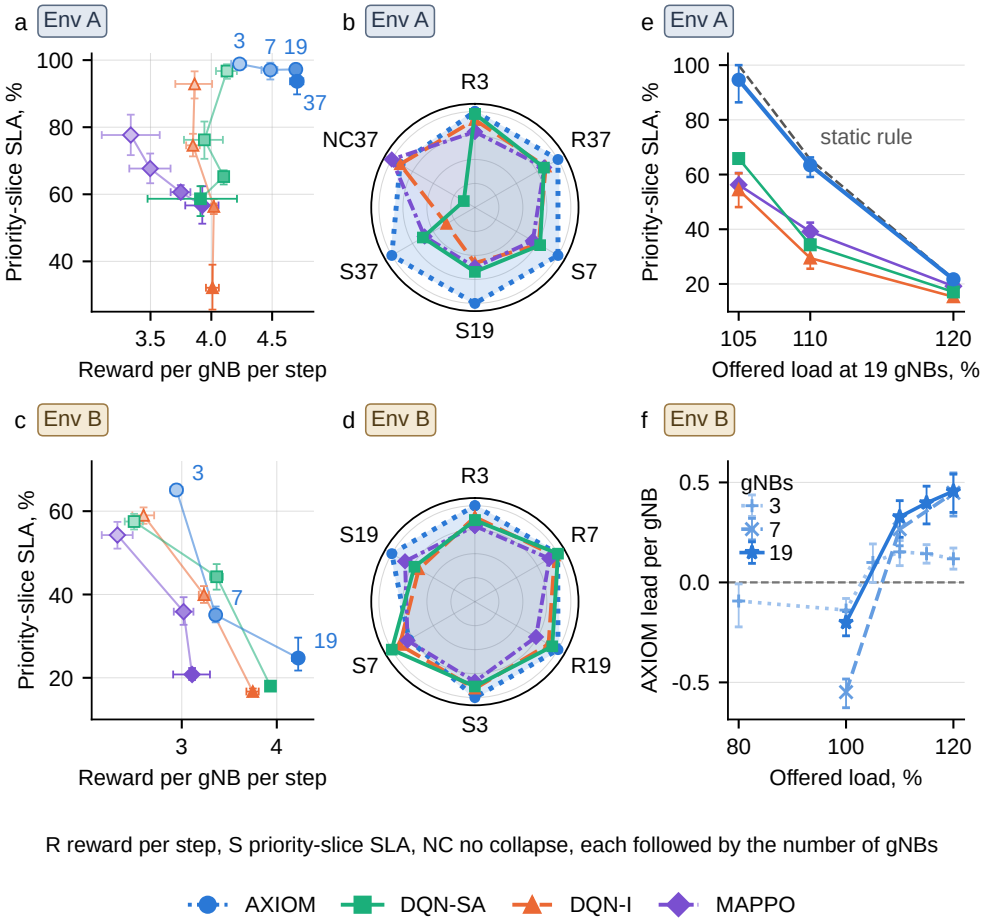


Fig. 8 AXIOM against the learned baselines, with 95% CIs. a, Env A and c, Env B reward per gNB against priority-slice compliance, light to dark by cluster size. b, d, Shares of the best learned policy. e, Env A compliance at 19 gNBs as load rises. f, AXIOM over the tuned rule in Env B as load shifts

6.6 Reproducibility and Fresh-Seed Verification

We checked the results in three ways, with every pass criterion fixed before the run it judged. First, rerunning eleven reported settings in fresh processes reproduced every stored result exactly. Sixteen further reruns, from seven to thirty-seven gNBs in both environments, also matched exactly. The one exception is DQN-SA, which builds its network before setting the seed, and seeding it leaves AXIOM's leads unchanged.

Second, four further runs retrained every policy on fresh seeds, 10 to 40 per setting, and 81 of their 90 pre-stated checks held. The nine checks that failed concern secondary margins rather than any headline result. On fresh seeds the DQN baselines reached up to 83.3% compliance at seven gNBs, above the 80% bound we had set, and AXIOM's small lead over DQN-SA in Env A at three gNBs did not recur. A fifth fresh-seed

run held all three of its checks at thirty-seven gNBs on 20 seeds. A sixth held all four in Env B at nineteen gNBs on 15 seeds, as Section S7 of OR1 reports. Section S2 of OR1 compares each reported difference from a learned baseline with its fresh-seed counterpart, and 35 of the 38 whose CI excludes zero hold.

Third, a script written independently of the analysis pipeline, with its own log reader and Holm correction, recomputes the numbers in every results table and the key results sentences from the raw logs. In the current version, all 625 of its checks pass.

6.7 Federation, Privacy, Disruption, and Live Deployment

Federated training costs AXIOM little reward per step on average. Pooled over all 30 federated seeds, ten of them fresh, the cost is $+0.491$ per step with 95% CI $[-0.08, 1.09]$ and $p = 0.14$. Even so, a cost of up to about one reward per step cannot be excluded. Per-step DP-SGD noise lowers reward by 0.77 to 1.25 per step without a significant drop, but its formal guarantee is weak, with ε of about 2,600 or more. Under gNB dropout and agent churn every policy loses reward significantly, and no gap between AXIOM and any baseline survives Holm correction at ten seeds. Demand spikes leave AXIOM's decisions largely intact, with block precision at 0.98, and Section S5 of OR1 gives the protocol. A live single-gNB OAI testbed, shown in Fig. 9, confirmed the control path that AXIOM would use, with 0.000% loss at one and at two UEs per slice, but it never ran short of capacity, as Section S3 of OR1 reports.

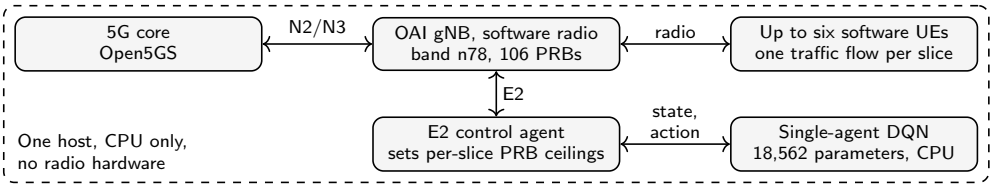


Fig. 9 Live single-gNB testbed on one host with a software radio. An offline-trained single-agent DQN stands in for AXIOM to check the control path

7 Discussion and Limitations

No learned baseline earns significantly more reward than AXIOM in any setting we tested, and its margin over the cheaper MAPPO is largest under time-varying load, coupling, and uneven per-gNB load. A rule known in advance, however, matches or beats AXIOM in steady conditions at every size, and AXIOM beats a tuned rule only when the load moves beyond the rule's tuning conditions. Its practical case is for operators who cannot hand-tune and re-tune rules as conditions change, rather than for a fixed, well-understood cluster.

Several limitations bound the findings of this paper. First, both of our simulators are synthetic rather than measured. They use a first-order coupling model with no radio channel, user mobility, or measured traffic traces, and the live testbed never reached contention. External validity on hardware under load therefore remains the

main open question. Second, we did not reimplement a published slice-admission algorithm, so our learned baselines are standard multi-agent methods and ablations of AXIOM rather than prior admission controllers. Third, attention added nothing over a per-node encoder, and a value-decomposition baseline matched AXIOM at three gNBs, so AXIOM's contribution is a design that scales rather than a new learning principle. Fourth, charging the SLA penalty per violating gNB, so that it grows with the cluster, changed neither AXIOM's compliance nor its standing against the static rule. Finally, the pre-registrations were committed to the authors' own version-control history rather than an external registry, and development work that changed no reported result is disclosed in Section S4 of OR1.

8 Conclusion

This paper presented AXIOM, a multi-agent slice admission controller with a shared graph-attention encoder and per-slice heads, evaluated under a protocol designed to make its results trustworthy. AXIOM's advantage over learned controllers grows with the cluster. Under static load it reaches 98.9% of an oracle rule's reward at three gNBs and keeps priority-slice compliance between 93.7% and 97.3% from seven to thirty-seven gNBs, while every learned baseline stays below 84%. At thirty-seven gNBs its lead over the best learned baseline reaches 35.0 points, and 36.5 points on 20 fresh seeds, with 7,782 parameters against up to 682,058.

Under time-varying load and coupling AXIOM earns 27% to 36% more reward than MAPPO at three and nineteen gNBs, so its extra cost pays most where the environment is dynamic. When load rises 10% to 20% above the conditions a tuned rule was set for, AXIOM overtakes that rule at nineteen gNBs by 6.2 to 8.7 reward per step without retraining. Simple rules still set the bar in other settings. A static rule known in advance does better under steady load and at nineteen gNBs under coupling, and a tuned rule beats every learned policy at its tuning conditions, as Section 7 discusses. The main findings held on fresh seeds never used before.

More broadly, the evaluation safeguards show that learned admission control should be judged against trivial and tuned rules. Its metrics should be tied to the objective, and its results should hold on independent seeds under a pre-registered analysis.

Three directions for future work follow from these limits. First, a live multi-gNB deployment of AXIOM would test its gains under real contention. Second, giving the learner the queue-threshold structure that the tuned rules exploit may close the remaining gap to them. Finally, adding noise to the per-round model updates rather than to every step would make the formal privacy guarantee meaningful.

Supplementary information. Online Resource 1 is a PDF that gives the full verification results, the live testbed check, additional setup details, and additional results.

Acknowledgements. This work was supported by the Fundamental Research Grant Scheme (FRGS) of the Ministry of Higher Education (MOHE), Malaysia, under grant FRGS/1/2024/ICT06/TAYLOR/02/1.

Statements and Declarations

The following statements should be understood to apply to this manuscript unless otherwise indicated.

- Funding: This work was supported by the Fundamental Research Grant Scheme (FRGS), Ministry of Higher Education (MOHE), Malaysia, under grant number FRGS/1/2024/ICT06/TAYLOR/02/1.

- Competing interests: The authors declare no competing interests.
- Ethics approval and consent to participate: Not applicable, since this study involves no human participants, animal subjects, or identifiable patient data. All reported results are either offline simulation campaigns or measurements from a laboratory radio testbed.
- Consent for publication: Not applicable, since this manuscript contains no personal data or identifying information about any individual.
- Data availability: The experiment logs, checkpoints, and campaign result files underlying this paper's findings are available from the corresponding author on reasonable request. They are not yet publicly archived because the research is ongoing and the dataset is not yet complete.
- Materials availability: Not applicable, since this study involves no physical or biological materials.
- Code availability: The framework and experiment scripts used to produce this paper's results, including the reproducibility manifest and analysis scripts used to independently re-derive every table and figure in this paper, are available from the corresponding author on reasonable request.
- Author contributions: M. P. Kunasegran: conceptualisation, methodology, software, investigation, writing (original draft). W. L. Pang: supervision, writing (review and editing). S. K. Phang: supervision, writing (review and editing).

References

[1] Abadi M, Chu A, Goodfellow I, et al (2016) Deep learning with differential privacy. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS), pp 308–318, https://doi.org/10.1145/2976749.2978318

[2] Ahmadi M, Moayyedi A, Sulaiman M, et al (2024) Generalizable 5G RAN/MEC slicing and admission control for reliable network operation. IEEE Trans Netw Service Manag 21(5):5384–5399. https://doi.org/10.1109/TNSM.2024.3437217

[3] AlQahtani SA (2025) Dynamic slice allocation in 5G cloud RAN. Wireless Personal Communications 143(3-4):427–467. https://doi.org/10.1007/s11277-025-11835-y

[4] Amiri E, Wang N, Shojafar M, et al (2024) Edge-AI empowered dynamic VNF splitting in O-RAN slicing: A federated DRL approach. IEEE Communications Letters 28(2):318–322. https://doi.org/10.1109/LCOMM.2023.3341302

[5] Bega D, Gramaglia M, Banchs A, et al (2020) A machine learning approach to 5G infrastructure market optimization. IEEE Transactions on Mobile Computing 19(3):498–512. https://doi.org/10.1109/TMC.2019.2896950

[6] Bouroudi A, Outtagarts A, Hadjadj-Aoul Y (2025) GNN-based multi-agent DRL for energy-efficient multi-domain 6G resource orchestration. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 4457–4462, https://doi.org/10.1109/GLOBECOM59602.2025.11432814

[7] Bun M, Steinke T (2016) Concentrated differential privacy: Simplifications, extensions, and lower bounds. In: Theory of Cryptography Conference (TCC), pp 635–658, https://doi.org/10.1007/978-3-662-53641-4_24

[8] Doanis P, Spyropoulos T (2024) Sample-efficient multi-agent DQNs for scalable multi-domain 5G+ inter-slice orchestration. IEEE Transactions on Machine Learning in Communications and Networking 2:956–977. https://doi.org/10.1109/TMLCN.2024.3420268

[9] Fatehi K, Ghourtani MR, Sonee A, et al (2026) Interpretable attention-based multi-agent PPO for latency spike resolution in 6G RAN slicing. In: Proc. IEEE Int. Conf. Commun. (ICC), pp 1–6, https://doi.org/10.1109/ICC59461.2026.11586913

[10] Gong Y, Gong P, Jiang H, et al (2026) Intelligent resource allocation for multiaccess edge computing in 5G ultra-dense slicing network using federated multiagent DDPG algorithm. China Commun 23(1):273–289. https://doi.org/10.23919/JCC.fa.2022-0187.202601

[11] Hao X, She C, Yeoh PL, et al (2024) Hybrid-task meta-learning: A GNN approach for scalable and transferable bandwidth allocation. IEEE Transactions on Wireless Communications 23(12):19820–19835. https://doi.org/10.1109/TWC.2024.3487533

[12] Kamal YB, Abdulkafi AA (2026) Reinforcement learning double deep Q-network techniques for NextG network. Wireless Personal Communications 146(6):3175–3197. https://doi.org/10.1007/s11277-026-12048-7

[13] Khani M, Jamali S, Sohrabi MK, et al (2024) Slice admission control in 5G cloud radio access network using deep reinforcement learning: A survey. International Journal of Communication Systems 37(13):e5857. https://doi.org/10.1002/dac.5857

[14] Krishnamoorthy R, Gireesh Babu CN, Gite P, et al (2026) Federated learning for dynamic resource allocation in 6G network slicing. Wireless Personal Communications https://doi.org/10.1007/s11277-026-11943-3

[15] Kulkarni D, Venkatesan M, Kulkarni AV (2024) Actor critic based reinforcement learning for joint resource allocation and throughput maximization in 5G RAN slicing. Wireless Personal Communications 138(1):623–640. https://doi.org/10.1007/s11277-024-11526-0

[16] Kunasegran MP, Pang WL, Phang SK (2025) Optimising resource allocation in 5G networks: Balancing URLLC and eMBB traffic under gNB congestion. In: 2025 Multimedia University Engineering Conference (MECON), pp 1–6, https://doi.org/10.1109/MECON67253.2025.11277096

[17] Kunasegran MP, Pang WL, Phang SK (2025) SLA-aware DRL for joint slice admission control and load balancing in multi-service 5G RAN. In: 2025 9th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), pp 207–212, https://doi.org/10.1109/ICRAIE65839.2025.11239148

[18] Kunasegran MP, Pang WL, Phang SK (2026) Comprehensive review of optimization techniques for user-centric distributed network slicing in 5G networks. IEEE Access 14:114630–114648. https://doi.org/10.1109/ACCESS.2026.3716911

[19] Li F, Hao Y, Yang S, et al (2025) OACR^2: Online admission control and resource reservation for 5G slice networks with deep reinforcement learning. IEEE Transactions on Mobile Computing 24(8):7360–7376. https://doi.org/10.1109/TMC.2025.3548767

[20] Li Y, Wang H, Min G, et al (2026) A flexible and scalable multi-agent learning framework for dynamic RAN slicing in 6G native-AI networks. IEEE Transactions on Mobile Computing 25(5):7258–7273. https://doi.org/10.1109/TMC.2025.3642213

[21] Lim WYB, Luong NC, Hoang DT, et al (2020) Federated learning in mobile edge networks: A comprehensive survey. IEEE Communications Surveys & Tutorials 22(3):2031–2063. https://doi.org/10.1109/COMST.2020.2986024

[22] Ming Z, Yu H, Taleb T (2024) Federated deep reinforcement learning for prediction-based network slice mobility in 6G mobile networks. IEEE Transactions on Mobile Computing 23(12):11937–11953. https://doi.org/10.1109/TMC.2024.3404125

[23] Mohajer A, Mirzaei A, Darabi M, et al (2026) Joint SLA-aware task offloading and adaptive service orchestration with graph-attentive multi-agent reinforcement learning. IEEE Trans Netw Service Manag 23:3737–3754. https://doi.org/10.1109/TNSM.2026.3673188

[24] Ojijo MO, Ramotsoela D, Oginga RA (2024) Slice admission control in 5G wireless communication with multi-dimensional state space and distributed action space: A sequential twin actor-critic approach. Computer Networks 255:110878. https://doi.org/10.1016/j.comnet.2024.110878

[25] Phyu HP, Naboulsi D, Stanica R (2025) ICE-CREAM: Multi-agent fully cooperative decentralized framework for energy efficiency in RAN slicing. IEEE Transactions on Network and Service Management 22(2):1859–1873. https://doi.org/10.1109/TNSM.2024.3524503

[26] Polese M, Bonati L, D'Oro S, et al (2023) ColO-RAN: Developing machine learning-based xApps for open RAN closed-loop control on programmable experimental platforms. IEEE Transactions on Mobile Computing 22(10):5787–5800. https://doi.org/10.1109/TMC.2022.3188013

[27] Polese M, Bonati L, D'Oro S, et al (2023) Understanding O-RAN: Architecture, interfaces, algorithms, security, and research challenges. IEEE Communications Surveys & Tutorials 25(2):1376–1411. https://doi.org/10.1109/COMST.2023.3239220

[28] Rashid T, Samvelyan M, Schroeder de Witt C, et al (2020) Monotonic value function factorisation for deep multi-agent reinforcement learning. Journal of Machine Learning Research 21(178):1–51

[29] Raza MR, Natalino C, Öhlen P, et al (2019) Reinforcement learning for slicing in a 5G flexible RAN. Journal of Lightwave Technology 37(20):5161–5169. https://doi.org/10.1109/JLT.2019.2924345

[30] Shao Y, Li R, Hu B, et al (2021) Graph attention network-based multi-agent reinforcement learning for slicing resource management in dense cellular network. IEEE Trans Veh Technol 70(10):10792–10803. https://doi.org/10.1109/TVT.2021.3103416

[31] Sirma M, Kavak A, Inner AB (2025) Optimizing relay selection in D2D communication for next-generation wireless networks using multi-agent reinforcement learning: A novel approach. Wireless Personal Communications 140(3-4):945–969. https://doi.org/10.1007/s11277-025-11753-z

[32] Sulaiman M, Moayyedi A, Ahmadi M, et al (2023) Coordinated slicing and admission control using multi-agent deep reinforcement learning. IEEE Trans Netw Service Manag 20(2):1110–1124. https://doi.org/10.1109/TNSM.2022.3222589

[33] Sunehag P, Lever G, Gruslys A, et al (2018) Value-decomposition networks for cooperative multi-agent learning based on team reward. In: Proceedings of the 17th International Conference on Autonomous Agents and MultiAgent Systems (AAMAS), pp 2085–2087, https://doi.org/10.65109/jsrc7365

[34] Tao Z, Xu W, You X (2025) Digital twin-accelerated online deep reinforcement learning for admission control in sliced communication networks. IEEE Transactions on Communications 73(4):2490–2504. https://doi.org/10.1109/TCOMM.2024.3476430

[35] Tashman DH, Cherkaoui S (2026) Adversarial attacks in AI-driven RAN slicing: SLA violations and recovery. In: Proc. Int. Wireless Commun. and Mobile Computing (IWCMC), pp 1391–1396, https://doi.org/10.1109/IWCMC69287.2026.11580033

[36] Tsampazi M, D'Oro S, Polese M, et al (2023) A comparative analysis of deep reinforcement learning-based xApps in O-RAN. In: Proc. IEEE Global Communications Conference (GLOBECOM), pp 1638–1643, https://doi.org/10.1109/GLOBECOM54140.2023.10437367

[37] Tsampazi M, D'Oro S, Polese M, et al (2025) PandORA: Automated design and comprehensive evaluation of deep reinforcement learning agents for open RAN. IEEE Transactions on Mobile Computing 24(4):3223–3240. https://doi.org/10.1109/TMC.2024.3505781

[38] Wu Z, Pan S, Chen F, et al (2021) A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems 32(1):4–24. https://doi.org/10.1109/TNNLS.2020.2978386

[39] Yasin TV, Yu CM, Wang LC (2025) Differential privacy federated edge learning-assisted for securing RAN intelligent controller in O-RAN 6G communications. In: Proc. IEEE VTS Asia Pacific Wireless Commun. Symp. (APWCS), pp 1–5, https://doi.org/10.1109/APWCS67981.2025.11151868

[40] Yu C, Velu A, Vinitsky E, et al (2022) The surprising effectiveness of PPO in cooperative multi-agent games. In: Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track, pp 24611–24624, https://doi.org/10.52202/068431-1787

[41] Zangooei M, Golkarifard M, Rouili M, et al (2024) Flexible RAN slicing in Open RAN with constrained multi-agent reinforcement learning. IEEE Journal on Selected Areas in Communications 42(2):280–294. https://doi.org/10.1109/JSAC.2023.3336156

[42] Zhang H, Zhou H, Erol-Kantarci M (2022) Federated deep reinforcement learning for resource allocation in O-RAN slicing. In: Proc. IEEE Global Commun. Conf. (GLOBECOM), pp 958–963, https://doi.org/10.1109/GLOBECOM48099.2022.10001658